

UML Diagram - Sistem Pemesanan Tiket Online (Final)

1. USE CASE DIAGRAM

```
@startuml
left to right direction
skinparam packageStyle rectangle

actor Pembeli
actor Organizer
actor Admin
actor "Payment Gateway" as PG

rectangle "Sistem Pemesanan Tiket Online" {

    usecase "Registrasi & Login" as UC1
    usecase "Cari & Lihat Detail Event" as UC2
    usecase "Pilih Kursi & Jumlah Tiket" as UC3
    usecase "Checkout & Pembayaran" as UC4
    usecase "Lihat Status Pesanan" as UC5
    usecase "Download E-Tiket (QR Code)" as UC6
    usecase "Check-in dengan QR Code" as UC7
    usecase "Ajukan Refund" as UC8

    usecase "Buat Event Baru" as UC9
    usecase "Kelola Detail Event" as UC10
    usecase "Atur Kategori & Harga Kursi" as UC11
    usecase "Batalkan / Reschedule Event" as UC12
    usecase "Lihat Laporan Penjualan" as UC13
    usecase "Proses Refund Pembeli" as UC14

    usecase "Verifikasi Akun Organizer" as UC15
    usecase "Approve / Reject Event" as UC16
    usecase "Suspend Akun Pengguna" as UC17
    usecase "Lihat Laporan Transaksi" as UC18

    usecase "Proses Pembayaran" as UC19
    usecase "Kirim Notifikasi Status Bayar" as UC20
}
```

```
}
```

```
Pembeli --> UC1  
Pembeli --> UC2  
Pembeli --> UC3  
Pembeli --> UC4  
Pembeli --> UC5  
Pembeli --> UC6  
Pembeli --> UC7  
Pembeli --> UC8
```

```
Organizer --> UC1  
Organizer --> UC9  
Organizer --> UC10  
Organizer --> UC11  
Organizer --> UC12  
Organizer --> UC13  
Organizer --> UC14
```

```
Admin --> UC1  
Admin --> UC15  
Admin --> UC16  
Admin --> UC17  
Admin --> UC18
```

```
PG --> UC19  
PG --> UC20
```

```
UC4 --> UC19 : <<include>>  
UC19 --> UC20 : <<include>>  
UC12 --> UC14 : <<include>>  
UC6 ..> UC5 : <<extend>>
```

```
@enduml
```

2. COLLABORATION DIAGRAM (Kompleks - 1 Diagram)

Skenario Lengkap: Dari Pembuatan Event hingga Pembeli Menerima E-Tiket

```
@startuml  
skinparam object {  
    BackgroundColor LightYellow  
    BorderColor DarkOrange
```

```

    FontSize 11
}

' ===== OBJEK AKTOR =====
object ":Organizer" as ORG
object ":Admin" as ADM
object ":Pembeli" as PBL

' ===== OBJEK CONTROLLER =====
object ":EventController" as EC
object ":PesananController" as PC

' ===== OBJEK SERVICE =====
object ":ValidasiService" as VS
object ":ApprovalService" as AS
object ":KursiService" as KS
object ":PembayaranService" as PS
object ":TiketService" as TS
object ":NotifikasiService" as NS
object ":RefundService" as RS

' ===== OBJEK EKSTERNAL =====
object ":PaymentGateway" as PG

' ===== OBJEK DATA =====
object "event:Event" as EV
object "kursi:KategoriKursi" as KK
object "pesanan:Pesanan" as PES
object "tiket:Tiket" as TIK

' =====
' FASE 1: ORGANIZER MEMBUAT EVENT
' =====
ORG --> EC : 1. buatEventBaru(judul, tanggal, venue)
EC --> VS : 1.1. validasiDataEvent(data)
VS --> EC : 1.2. returnValidasiOK()
EC --> EV : 1.3. create(judul, tanggal, venue, status=DRAFT)
ORG --> EC : 2. aturKategoriKursi(kategori, harga, kuota)
EC --> KS : 2.1. tambahKategori(eventId, kategori, harga, kuota)
KS --> KK : 2.2. create(eventId, kategori, harga, kuota)
ORG --> EC : 3. submitEvent(eventId)
EC --> EV : 3.1. setStatus(MENUNGGU_APPROVAL)
EC --> NS : 3.2. kirimNotifikasiAdmin(eventId)
NS --> ADM : 3.3. notifikasiReviewEvent(eventId)

' =====
' FASE 2: ADMIN APPROVAL EVENT

```

```
' =====
ADM --> AS : 4. reviewEvent(eventId)
AS --> EV : 4.1. getDetailEvent(eventId)
EV --> AS : 4.2. returnDetailEvent()
ADM --> AS : 5. approveEvent(eventId)
AS --> EV : 5.1. setStatus(AKTIF)
AS --> NS : 5.2. kirimNotifikasiOrganizer(eventId, APPROVED)
NS --> ORG : 5.3. notifikasiEventAktif()
AS --> NS : 5.4. broadcastEventKePublik(eventId)
```

```
' =====
' FASE 3: PEMBELI MEMESAN TIKET
```

```
' =====
PBL --> PC : 6. cariDanPilihEvent(keyword)
PC --> EV : 6.1. getListEvent(filter)
EV --> PC : 6.2. returnListEvent()
PBL --> PC : 7. pilihKursi(eventId, kategori, jumlah)
PC --> KS : 7.1. cekKetersediaan(eventId, kategori, jumlah)
KS --> PC : 7.2. returnKursiTersedia()
PC --> PES : 8. create(pembeli, eventId, kursi, total)
PC --> KS : 8.1. reservasiSementara(kursiId, 30menit)
PC --> NS : 8.2. kirimTimerPembayaran(pesananId, 30menit)
NS --> PBL : 8.3. notifikasiDeadlineBayar()
```

```
' =====
' FASE 4: PEMBAYARAN
```

```
' =====
PBL --> PC : 9. konfirmasiCheckout(pesananId, metodeBayar)
PC --> PS : 9.1. inisiasiBayar(pesananId, metodeBayar, total)
PS --> PG : 9.2. requestPembayaran(amount, token, metode)
PG --> PS : 9.3. returnStatusBayar(sukses/gagal)
PS --> PES : 10. setStatus(LUNAS)
PS --> KS : 10.1. konfirmasiReservasiKursi(kursiId)
KS --> KK : 10.2. updateKuotaTersedia(kursiId)
PC --> TS : 11. generateETiket(pesananId)
TS --> TIK : 11.1. create(pesananId, qrCode, expiredAt)
TS --> NS : 11.2. kirimETiket(pembeli, tiket)
NS --> PBL : 11.3. notifikasiETiketSiap(etiket, qrCode)
NS --> ORG : 11.4. notifikasiTiketTerjual(eventId, jumlah)
```

```
' =====
' FASE 5: CHECK-IN DAN REFUND (jika event dibatalkan)
```

```
' =====
PBL --> TS : 12. checkIn(qrCode, eventId)
TS --> TIK : 12.1. validasiQRCode(qrCode)
TIK --> TS : 12.2. returnValidasiOK()
TS --> TIK : 12.3. setStatus(SUDAH_DIGUNAKAN)
```

```

TS --> NS : 12.4. kirimKonfirmasiCheckIn(pembeli)

ORG --> EC : 13. batalkanEvent(eventId, alasan)
EC --> EV : 13.1. setStatus(DIBATALKAN)
EC --> RS : 13.2. triggerRefundOtomatis(eventId)
RS --> PES : 13.3. getSemuaPesananAktif(eventId)
RS --> PS : 13.4. prosesRefundMassal(listPesananId)
PS --> PG : 13.5. requestRefund(amount, token)
PG --> PS : 13.6. returnStatusRefund(sukses)
RS --> PES : 13.7. setStatus(REFUND_SELESAI)
RS --> NS : 13.8. kirimNotifikasiRefund(listPembeli)
NS --> PBL : 13.9. notifikasiRefundBerhasil(amount)

@enduml

```

Ringkasan Fase:

Fase	Pesan	Aktor Terlibat	Keterangan
1. Buat Event	1 - 3.3	Organizer	Input data, atur kursi, submit ke Admin
2. Approval	4 - 5.4	Admin	Review, approve, broadcast ke publik
3. Pemesanan	6 - 8.3	Pembeli	Cari event, pilih kursi, reservasi sementara
4. Pembayaran	9 - 11.4	Pembeli, Payment Gateway	Bayar, generate e-tiket, notifikasi
5. Check-in & Refund	12 - 13.9	Pembeli, Organizer	Scan QR atau proses refund jika dibatalkan

3. STATE MACHINE DIAGRAM (Kompleks - 1 Diagram)

Siklus Hidup Terintegrasi: Event & Pesanan dalam Satu Sistem

```

@startuml
skinparam state {
    BackgroundColor LightYellow
    BorderColor DarkOrange
    ArrowColor DarkRed
}

```

```

    FontSize 11
}

' =====
' SIKLUS HIDUP EVENT (kiri)
' =====

state "SIKLUS HIDUP EVENT" as SEVENT {
    [*] --> Draft : Organizer buat event baru

    Draft --> MenungguApproval : Organizer submit event
    MenungguApproval --> Aktif : Admin approve event
    MenungguApproval --> Ditolak : Admin reject\n(data tidak valid)
    Ditolak --> Draft : Organizer perbaiki & submit ulang
    Ditolak --> [*] : Organizer tidak lanjut

    Aktif --> TiketDijual : Minimal 1 tiket terbeli
    TiketDijual --> TiketHabis : Semua kuota terjual
    TiketDijual --> Berlangsung : Hari H dimulai
    TiketHabis --> Berlangsung : Hari H dimulai
    Berlangsung --> EventSelesai : Event berakhir
    EventSelesai --> [*]

    Aktif --> EventDibatalkan : Organizer batalkan
    TiketDijual --> EventDibatalkan : Organizer batalkan\nsetelah ada pembeli
    EventDibatalkan --> RefundMassal : Sistem trigger\nrefund otomatis
    RefundMassal --> EventSelesai : Semua refund berhasil
}

' =====
' SIKLUS HIDUP PESANAN (kanan)
' =====

state "SIKLUS HIDUP PESANAN" as SPESANAN {
    [*] --> PesananDibuat : Pembeli pilih kursi & checkout

    PesananDibuat --> MenungguPembayaran : Sistem konfirmasi pesanan\n(timer 30 men
    MenungguPembayaran --> Dibayar : Pembayaran berhasil
    MenungguPembayaran --> Kedaluwarsa : Timer habis\ntanpa pembayaran

    Kedaluwarsa --> [*] : Kursi dilepas ke sistem

    Dibayar --> TiketAktif : E-tiket & QR Code diterbitkan
    TiketAktif --> CheckIn : Pembeli scan QR\ndi pintu masuk venue
    CheckIn --> TiketDigunakan : Verifikasi QR berhasil
    TiketDigunakan --> PesananSelesai : Event selesai
    PesananSelesai --> [*]

    TiketAktif --> PengajuanRefund : Pembeli ajukan refund\n(event dibatalkan)

```

```

    PengajuanRefund --> RefundDisetujui : Admin/Organizer\napprove refund
    PengajuanRefund --> RefundDitolak : Admin tolak\n(tidak memenuhi syarat)
    RefundDitolak --> TiketAktif : Tiket kembali aktif
    RefundDisetujui --> ProsesPengembalianDana : Sistem proses\npengembalian dana
    ProsesPengembalianDana --> RefundSelesai : Dana berhasil\ndikembalikan
    RefundSelesai --> [*]
}

' =====
' RELASI ANTAR SIKLUS
' =====

EventDibatalkan --> PengajuanRefund : Event dibatalkan\nmemicu refund pesanan
Aktif --> PesananDibuat : Event aktif\nmemungkinkan pemesanan
TiketDijual --> TiketAktif : Pembayaran sukses\nmengaktifkan tiket

@enduml

```

Penjelasan State:

State	Objek	Keterangan
Draft	Event	Event baru dibuat, belum disubmit
Menunggu Approval	Event	Menunggu review Admin
Aktif	Event	Disetujui, tiket bisa dibeli
Tiket Dijual	Event	Sudah ada pembeli
Tiket Habis	Event	Semua kuota habis
Berlangsung	Event	Hari H sedang berjalan
Event Dibatalkan	Event	Organizer batalkan event
Refund Massal	Event	Trigger refund ke semua pembeli
Pesanan Dibuat	Pesanan	Pembeli checkout, belum bayar
Menunggu Pembayaran	Pesanan	Timer 30 menit berjalan
Dibayar	Pesanan	Pembayaran dikonfirmasi
Tiket Aktif	Pesanan	E-tiket sudah diterbitkan
Check-in	Pesanan	QR Code sedang di-scan

Tiket Digunakan	Pesanan	Berhasil masuk venue
Pengajuan Refund	Pesanan	Pembeli minta refund
Refund Selesai	Pesanan	Dana kembali ke pembeli

KETERKAITAN KETIGA DIAGRAM

	Use Case	Collaboration	State Machine
Menjawab	Siapa & apa fiturnya?	Bagaimana alur lengkapnya?	Bagaimana status berubah?
Aktor	Semua aktor	Organizer, Admin, Pembeli, PG	Event & Pesanan (objek)
Contoh link	UC4: Checkout & Pembayaran	Fase 4: Pesan 9-11.4	Pesanan: Dibayar → Tiket Aktif

Alur cerita: Use Case mendefinisikan **apa yang bisa dilakukan** → Collaboration menunjukkan **bagaimana 4 aktor berinteraksi** dalam 5 fase proses bisnis → State Machine melacak **bagaimana status Event dan Pesanan berubah** sebagai dampak dari interaksi tersebut.

CATATAN

- Render di: <https://www.plantuml.com/plantuml/uml/>
- Copy masing-masing blok @startuml ... @enduml lalu paste